

Trabalho 1 - Raw Sockets

Kauê de Amorim Silva

Otto Schmidt

1 O que foi feito

O sistema foi desenvolvido utilizando a linguagem Go, sendo constituído por dois binários principais: o `pacman_client` e o `pacman_server`.

- **Servidor:** Responsável por inicializar e gerenciar a matriz 40x40 do labirinto (criada a partir de um arquivo `.csv`), aplicar a movimentação dos fantasmas, contabilizar rodadas e transferir a visão de mapa apropriada e os eventuais arquivos (texto, imagem e vídeo).
- **Cliente:** Responsável por captar a intenção do usuário no terminal, convertê-la em um pacote e enviá-la ao servidor. A tela é renderizada utilizando a biblioteca gráfica *Bubbletea*, sendo o cliente responsável também por reconstituir e apresentar os arquivos recebidos como prêmios ou colisões.

2 Como foi feito

A implementação do sistema foi dividida em duas frentes principais: o Protocolo de Comunicação e a Lógica do Jogo.

2.1 Protocolo de Comunicação

As principais decisões técnicas que viabilizaram a comunicação foram:

1. **Cálculo de Verificação de Redundância Cíclica (CRC-8):** Utilizou-se o polinômio 0x07. Buscando reduzir o tempo de processamento, otimizou-se a rotina geradora de CRC utilizando uma *Lookup Table* com 256 valores, inicializada em tempo de compilação. A assinatura de integridade passou a ser calculada realizando apenas operações *XOR* sobre a tabela por byte analisado, evitando operações pesadas de divisão polinomial bit a bit em tempo real.
2. **Controle de Fluxo e Timeout Exponencial:** O timeout inicial é de 500ms. Durante a desconexão, cada timeout falho dobra o tempo de espera (com limite máximo de 4s), prevenindo congestionamentos na interface. Estipulou-se um máximo de 50 retentativas para evitar encerramento antecipado, o que mantém o jogo funcional caso um cabo seja reconectado. Além disso, implementou-se detecção de duplicatas (baseada no número de sequência) que reenvia o último *ACK* caso o mesmo pacote seja recebido.
3. **Transmissão de Arquivos:** Para viabilizar o fluxo de arquivos pesados (como imagens `.jpg` e vídeos `.mp4`), os dados são fatiados em múltiplos pacotes portando no máximo 31 bytes de conteúdo por quadro. Uma mensagem do tipo *End* sinaliza o fim da transmissão, delegando a reconstrução do artefato à máquina receptora de forma assíncrona, que o salva em `/tmp` e abre com `xdg-open`.

4. **Padding:** Quando um pacote (mensagem após empacotamento) era muito pequeno (menor que 14 bytes), ele era descartado pelos equipamentos de rede. Para contornar isso, passou-se a preencher esses pacotes curtos com bytes nulos.
5. **VLAN:** Se o pacote possuísse nos bytes de offset 13 os valores 0x88 ou 0x81, as interfaces de rede possuíam uma chance muito alta de tratar o tráfego seguindo o protocolo IEEE 802.1Q ou 802.1AD (*VLAN/QinQ*). Se tal byte de Ethertype for encontrado, o programa adiciona um byte com valor 0xFF para quebrar o campo, o qual é removido pela máquina de destino.

2.2 Lógica do Jogo

1. **Movimentação dos Fantasmas:** Os fantasmas possuem comportamentos distintos ao colidir com paredes: o Vermelho usa a regra da mão esquerda; o Azul usa a da mão direita; o Verde alterna entre esquerda e direita a cada colisão; e o Amarelo escolhe uma direção aleatória.
2. **Geração do Mapa:** Para evitar que o pacman, fantasmas e moedas sejam gerados em posições inacessíveis do cenário (por exemplo, dentro das letras P e R do mapa da UFPR), é possível inserir o caractere - no arquivo .csv do mapa. Este caractere impede a geração de entidades naquela posição, porém fica invisível visualmente para os jogadores.
3. **Janela de Logs:** Para imprimir um log na janela separada, as mensagens alimentam uma *Goroutine* que escreve de forma não-bloqueante em um *pipe* nomeado (/tmp/pacman_pipe). No início do programa, é invocada uma nova janela de terminal (ptyxis) encarregada apenas de ler esse *pipe*.

3 Testes Realizados

1. **Ambiente Local (lo):** Testes iniciais de transmissão de mensagens na interface loopback.
2. **Ambiente Virtual (vnet):** Testes da implementação do protocolo de rede finalizado, principalmente o transporte de arquivos e timeouts, numa interface virtual para comunicação com uma máquina virtual.
3. **Conexão Física a Cabo:** Validação final do protocolo e do jogo, retirando o cabo repetidamente para atestar o funcionamento das retransmissões.

4 Resultados e Conclusão

O sistema final apresenta alta estabilidade. O controle de erros e as retransmissões operam em conjunto para contornar perdas eficientemente. Contudo, o protocolo apresenta certa lentidão ao transportar arquivos grandes, pois para transmitir 31 bytes são necessárias 2 mensagens (uma de dados e um ACK), aumentando a latência e diminuindo a velocidade de transmissão.